Web Application Architecture: MVC and the Separation of Concerns

Introduction to Web Technology — Week 9

Index

1. The Hook: One Click, Three Roles — Why pressing "Add to Cart" is secretly a three-person team effort.
2. What Is MVC? — A 1970s idea that refuses to retire.
3. How the Three Parts Connect — Office gossip rules: nobody talks except through the Controller.
4. The Model — The workhorse who knows everything and shows nothing.
5. The View: Presentation Layer — The pretty face that only speaks when spoken to.
6. The Controller — The middle manager who delegates everything.
7. Worked Example: A To-Do List — Buying milk, one architectural layer at a time.
8. Real-World Example: Online Shopping — The same trio, now with a credit card.
9. Separation of Concerns — Why designers and database engineers can share an office without bloodshed.
10. Frontend and Backend — Who lives in the browser and who lives behind the curtain.
11. The Browser and Server Split — A border wall with an HTTP checkpoint.
12. MVC in a Real Framework — Spot the trio hiding in three humble files.
13. Workflow of a Modern Web Application — A page load is just a short, polite conversation.
14. Anatomy of a Request — Eight hops from click to pixels.
15. Practical Example: Tracing a Request — Following student 42's profile click under the microscope.
16. Discussion: Alternatives, Trends and Limits — MVC's rivals, its heirs, and its bad habits.
17. Recap — The whole chapter on one coffee break.
18. Practice Questions — Five problems that will not be solved by memorising.

MVC Architecture: One Click, Three Roles

Clicking **Add to Cart** changes the cart count through three cooperating components:

- **Model:** Manages data and business logic (e.g., stock and cart updates).
- **View:** Displays the interface and updates the cart badge.
- **Controller:** Handles user events and coordinates the Model and View.

Context

The **client–server paradigm** separates the front end and back end, communicating through HTTP request–response. **MVC** organises an application's internal structure by separating data, presentation, and coordination.

Events & Scalability

Typing, ticking a checkbox, or clicking a button generates an **event**, triggering effects such as displaying a page, executing logic, or modifying database records. Modular architecture and **separation of concerns** make large applications easier to maintain and scale for multiple clients.

Learning Objectives

- Understand MVC, the View, and the Controller.
- Learn separation of concerns and its benefits.
- Explore alternative architectures and emerging approaches.
- Understand the complete request workflow, from user event to resulting effect.

Restaurant Analogy

Customer = User | Waiter = Controller | Kitchen = Model | Presentation = View. The waiter coordinates, the kitchen manages data and rules, and the presentation displays the result—each has a distinct responsibility.

What Is MVC?

MVC, short for **Model–View–Controller**, is a way of splitting a program into **three cooperating parts, each with one job**. The three compartments are distinct, do not overlap, and have clear demarcations of responsibility, yet they cooperate to make one application work.

Part	Its one job
Model	Owns the data and the rules for working with it
View	Shows results to the user; it only cares about display, not where the data came from
Controller	Takes input from the user and tells the Model what to do and the View what to show

The Three Parts in a Little More Detail

- **Model.** Think of the database where your entire data is stored. Inserting new data, deleting outdated data, and querying or retrieving data all fall under the Model's scope.
- **View.** This is the presentation. Whether the output appears on a phone or a laptop depends on screen size, and that is handled by how you design the front end with HTML, CSS and JavaScript. *View* is the architectural name for the compartment that keeps all presentation concerns in one place.
- **Controller.** It takes input from the user, tells the Model what to do, and tells the View what to show. In short, it **coordinates** between the Model and the View.

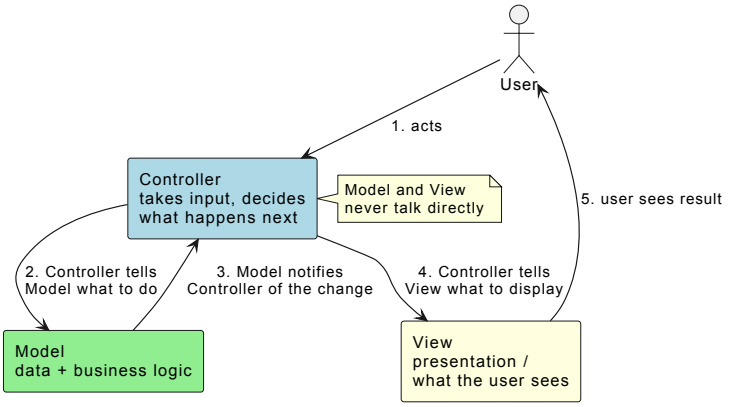
History and Present Use

The pattern dates back to **Smalltalk-80 at Xerox PARC in the late 1970s**, where it was advocated for large-scale solutions, and it **still shapes how most web frameworks are organized today**.

Framework	Well-known users / domains
Ruby on Rails	GitHub, Shopify, Airbnb
Django	Instagram, Disqus
Spring MVC (Java-based)	Broad usage in banking, insurance, e-commerce platforms and large corporate applications
ASP.NET MVC	Broad usage in banking, insurance, e-commerce platforms and large corporate applications

How the Three Parts Connect

Every user action travels a loop with five steps. **(1)** The user *acts* on some UI element: a click, typed text, or even a voice command if a speech API is used. This triggers the **Controller** first, which takes the input and decides what happens next. **(2)** The Controller *tells the Model what to do*. The Model owns the data and the methods (the business logic), so it alone decides how to perform the request, for example how to query or retrieve data. **(3)** Once the work is done, the Model *notifies the Controller of the change*. Note that it does not ship the data back by itself; it only reports that the task is complete. **(4)** The Controller then *tells the View what to display*, which is where presentation begins. **(5)** The user *sees the result*.



The golden rule is that **the Model and the View never talk directly**. Every communication between them is routed through the Controller: user → Controller → Model → Controller → View → user. In a responsive web application this loop repeats for *every* change, because the Controller is the main brain that routes traffic back and forth and controls how the user interacts with the application.

Splitting a program this way keeps each part focused on one job. MDN calls this a **"separation of concerns"**:

- the Model deals with the **data and its processing**,
- the View deals with **presentation**: the styling and design of how the output of the logic reaches the user,
- the Controller **routes** the communication between them.

> **Note for Review — Pro-Tip:** Exams love the question "which component talks to which?" Remember that the Model *notifies* the Controller (it does not push data to the View), and the Model and View have **no direct link**. In the original Smalltalk-style MVC the View may observe the Model directly; the web-style variant taught here sends everything through the Controller, so answer in that variant.

The Model

The Model is the part of the program that actually does the work. In MDN's phrase, it is **the workhorse**.

- **Owns the data:** in-memory objects, files, or records fetched from a database. Anything that is data in your application lives here.
- **Owns the rules:** validation, calculations, and whatever logic defines *correct behavior*. The entire business logic is part of the Model.
- A well-built Model **does not know or care how its data will be displayed, or by whom**. It only deals with data and logic, and who presents the output in what format is none of its concern.

Shopping-cart illustration. In an app such as Amazon, Flipkart or Myntra, the Model knows *which items are in the cart* and *what they cost*. Each item has an associated price (data). Summing up the total, applying a coupon, and adding shipping charges are calculations (rules). All of it belongs to the Model. How the cart list is drawn on screen, whether with check boxes, radio buttons or anything else, is nothing the Model knows about.

> **Note for Review — Pro-Tip:** Validation (for example "quantity must be at least 1") belongs to the **Model**, not to the Controller or the View. Remember that the Model defines what *correct* means.

The View: Presentation Layer

The View is responsible for **one thing: showing the current state to the user**. The current state is whatever the logic has produced around the data, and the View presents it.

- **Reads, never writes:** a well-designed View displays data but does not change it directly. Changing data is the Model's job.
- **HTML shaped by CSS:** on the web, the View is usually HTML shaped by CSS, the structure and style the browser renders. HTML provides the template, and CSS provides the styling flexibility to present the data in whatever fashion you want.
- **Passive by design:** the View waits to be told what to display; it does not decide when to update itself.
- **Many Views, one Model:** a mobile app screen and a web page can present the **exact same Model** without either one touching the other's code. Consider Amazon on an Android phone and Amazon in a browser: both draw from the same Model, but their Views differ greatly because of screen size, resolution and other factors we met in HTML and CSS.

> **Note for Review — Pro-Tip:** "Passive" does not mean "useless". It means the View never initiates a change. An exam answer such as "the View updates itself when the data changes" is **wrong** in this architecture; the Controller tells it to update.

The Controller

The Controller is the **go-between**: it takes input from the user and decides what happens next.

- **Reads input:** a click, a form submission, a keystroke.
- **Updates the Model:** by calling its methods, **never by reaching into its data directly**. The scope of updating data belongs to the Model.
- **Updates the View:** once the Model has changed, it tells the View what to display.
- The Controller **does not implement business rules itself** and **does not decide how something looks**. It only **routes**.

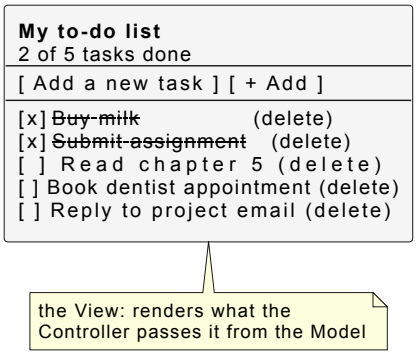
Updating is a continuous process, not a one-time event. Suppose you add five items to a cart: every addition changes the Model, and every time the Controller tells the View to refresh. If you remove an item, the same loop runs again, so the screen always reflects the latest state. It is a very interactive process.

	Reads / Writes	Must not know about
Model	Reads and writes the data	How the data is displayed, or by whom
View	Reads the data, never writes it	Where the data came from
Controller	Calls Model methods, tells View what to show	Business rules; how something looks

Worked Example: A To-Do List

Consider a simple task-management application, the kind that many of us use daily to remember what to do. Your phone almost certainly has one by default (a notes app) and you can install fancier ones that link to a calendar or email. Here we study the simplest version.

The screen shows a heading, **My to-do list**, with a counter saying *2 of 5 tasks done*. Below it is a text box labelled "Add a new task" with an **Add** button. Under that is the list of tasks, each with a check box and a delete button: two are ticked and struck through (Buy milk, Submit assignment), and three are pending (Read chapter 5, Book dentist appointment, Reply to project email). Everything you *see* here is the **View**. Behind it, a data store keeps the list of tasks and a Boolean "done" flag for each one, which gets toggled when you tick a task.

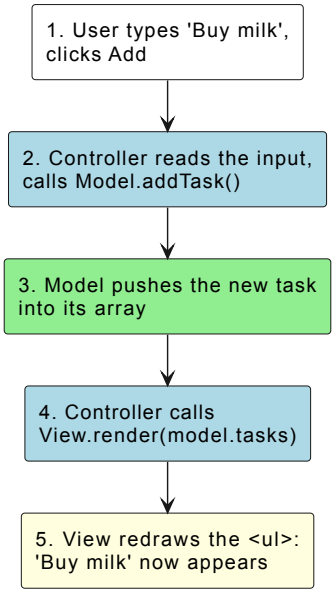


The Three Roles in This App

Role	What it holds or does
Model	`tasks = [{text: "Buy milk", done: false}]`, plus the methods `addTask(text)` and `toggleTask(id)`
View	Renders a `<ul>` with one `<li>` + checkbox per task in the array: `render(tasks)`
Controller	Listens for the Add button and checkbox clicks; calls Model methods, then `View.render()`

Step by Step: Adding "Buy milk"

The user types **Buy milk** and clicks **Add**. The Controller, which works as an *event handler* listening for the Add button and checkbox clicks, reads the input and calls `Model.addTask()`. Inside the Model the task is defined as a key–value pair (typically JSON-like): `text` is the key and "Buy milk" is the value, and `done` starts as `false` because a newly typed task is not done yet. The Model pushes the new task into its array. The Controller then calls `View.render(model.tasks)`, which is a status update saying "the data has changed, please present it accordingly". The View redraws the `<ul>` and "Buy milk" now appears in the list.



Which Decisions Live Where

- **The task text.** In this app the user may type *any* text, because everybody's life is different. Another design could offer a drop-down menu with a finite set of tasks to choose from, or give every item a priority state such as *low* or *medium*. All of these choices (what data exists, what the user is allowed to do with it) are **Model** logic.
- **The data structure.** The Model above uses an array, but it could use a list, a linked list or a database. Because the rest of the app only calls Model methods, this choice stays hidden.
- **The look.** A check box for "done", the Add button on the right-hand side, the list layout: all of this is the **View**. The button could just as well have been placed elsewhere.
- **The routing.** Listening for events and sequencing "update Model, then refresh View" is the **Controller**.

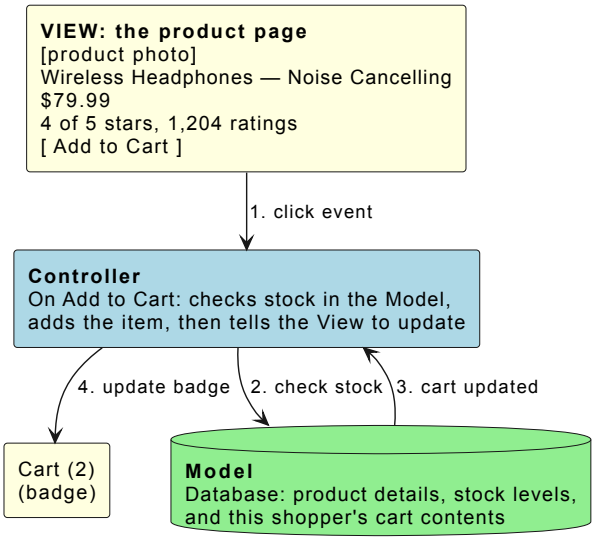
> **Same three roles as any MVC app. Only the data and the markup change with the domain.**

Real-World Example: Online Shopping

Back to the hook: clicking **Add to Cart** on any shopping site fires all three roles at once. Compare the earlier to-do list, where the Model handled data and rules, the View handled presentation, and the Controller mediated. In every interaction the Model updates the data and the View presents it then and there, in a cycle.

- **The View** is the product page itself: the photo, title, price, rating, and the button you clicked. It also shows the current state of the cart, such as a badge saying *Cart (2)*.
- **The Model** is the store's database: how many are in stock, what this item costs, what is already in your cart (and other product data such as ratings).
- **The Controller** is the code behind the button. It **checks the Model** (is the item in stock?), **updates it** (adds the item to the cart), and **tells the View what changed**. If no stock is available, the button can be disabled so the item cannot be added.

In the diagram, the click on **Add to Cart** is the event that reaches the Controller. The Controller asks the Model to check the stock, the Model reports that the cart was updated, and the Controller finally tells the View to update the cart badge. The Model and View never exchange a message. Swap "shopping cart" for a **playlist** (a music player such as Spotify), an **inbox** or a **timeline**, and the same three-way split still applies.



> **Same three roles as the abstract diagram.** Model, View and Controller are just organised around a shopping cart instead of a to-do list.

Separation of Concerns

MVC is one instance of a bigger idea: **separation of concerns**, which means keeping each part of a system focused on a single job so that parts do not overlap or interfere with each other. Data and logic live in one place, presentation in another. Two properties describe how well this is done.

Cohesion does not mean the parts are unrelated. They are cohesive in the sense that they all serve one application's common purpose, while each keeps a non-overlapping scope. They cannot be completely independent, since the Controller connects them, but how you encode the scope of each box should not create dependencies between the boxes.

Property	Meaning	Goal
Cohesion	Each component should serve one clear purpose	**High** cohesion inside each part
Coupling	Components should depend on each other as little as possible	**Low** coupling between parts

Why It Pays Off

When concerns are separated well, **a change to how something looks should not force a change to how it is calculated**. This is what gives you a scalable solution that survives growth from a small business to a very large enterprise, since complex logic can be encoded efficiently and each part can evolve alone.

- A **designer can redesign** the page's HTML and CSS, adopt new technology, change colours, add animation, or even move to a VR presentation, while a **backend engineer rewrites a database query**, **on the same day, without stepping on each other**.
- You can change the database technology: from a relational database to MongoDB, or adopt a graph database such as Neo4j, or an object database. You can deploy to a new server as the business grows. If concerns are separated, none of this changes anything in the presentation.

> **Note for Review — Pro-Tip:** Memorise the pair as **high cohesion, low coupling**. A common wrong answer is "the parts must be completely independent". They cooperate, but they must not *depend on each other's internals*.

Frontend and Backend

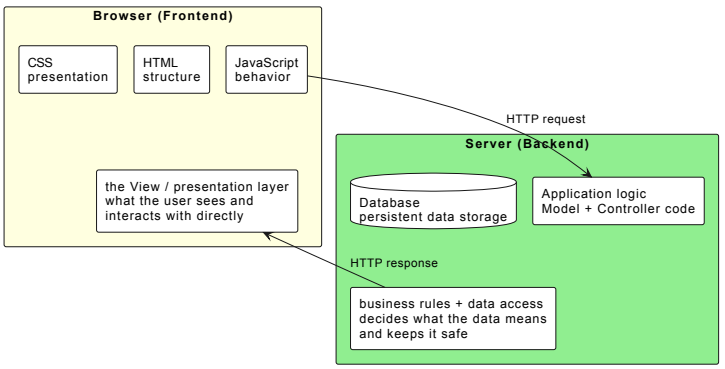
MVC maps onto the client–server world, though not perfectly.

- The user has no direct control over the backend; the application logic and business rules run there.
- The two communicate over **HTTP**: the frontend *asks*, the backend *answers*.
- The mapping is not as straightforward as it looks: **modern apps split some Controller logic across both** sides. Depending on the scale and logic of your application, you decide how to divide work between client and server.

	Frontend	Backend
Runs	In the browser	On the server
Made of	HTML, CSS and JavaScript that the user directly sees and touches	Application logic, business rules, and the database
MVC role	Roughly the **View**	Usually houses the **Model** and **Controller**
Typical tools	Plain HTML, or a JavaScript framework like React	Frameworks like Express, Django and Ruby on Rails

The Browser and Server Split

In the browser, **HTML** decides the structure, **CSS** deals with presentation, and **JavaScript** deals with behavior. Together they form the View, the presentation layer: what the user sees and interacts with directly. When any UI interaction occurs, an **HTTP request** goes to the server, and the Controller is the first interface it meets. The Controller in turn invokes the Model, which means the application logic and the database, where business rules and data are held. Finally, with the Controller's help, the View is updated through the **HTTP response**. The server decides what the data means and keeps it safe. Because the presentation layer and the data/business-logic layer sit on different sides of this boundary, **they can change independently**.



MVC in a Real Framework

Real frameworks implement the same split, just under different names. Our to-do app looks like this in a typical Python web framework:

MVC role	File	What it is
Model	`models.py`	A data class
Controller	`routes.py`	A route handler
View	`tasks.html`	A template

`models.py` is the Model. It defines what a task *is*: a `Task` class with a `text` field (a character field) and a `done` field (a Boolean that defaults to `False`). It talks only about data and the rules for the data, and nothing about screens or clicks.

```
class Task(Model):
    text = CharField()
    done = BooleanField(default=False)
```

`routes.py` is the Controller. The decorator `@app.route("/tasks/add")` ties a URL path to the function `add_task`. The handler reads the submitted text from the request form, calls the Model's `Task.create(...)` to store it, and then asks the framework to render `tasks.html`. It decides *when* to call the Model and *which* View to show; it does not implement the rules or the look itself.

```
@app.route("/tasks/add")
def add_task(request):
    Task.create(text=request.form["text"])
    return render("tasks.html")
```

`tasks.html` is the View. It is a template: a `<ul>` list in which the loop `{% for t in tasks %}` produces one `<li>` per task, and `{{ t.text }}` is a placeholder filled with the task's text. It only displays what it is handed.

```
<ul>
{% for t in tasks %}
  <li>{{ t.text }}</li>
{% endfor %}
</ul>
```

Most frameworks never say "Model", "View" or "Controller" out loud, **but the split is there**: a **data class**, a **route handler**, and a **template**.

> **Note for Review — Pro-Tip:** Naming can mislead you. Django, for example, calls its pieces *Model–Template–View*, and its function called a "view" plays the role of the **Controller**. Always identify a component by the *job it does*, not by its name.

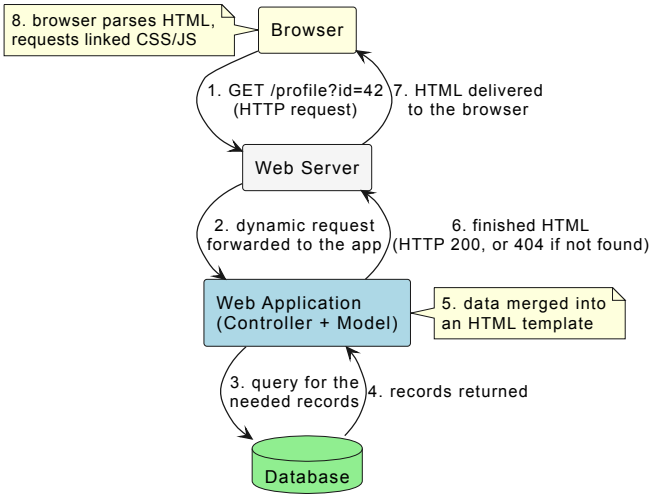
Workflow of a Modern Web Application

Every dynamic page load is a **short conversation between the browser and one or more servers**. A *dynamic* page is one that interacts with the user and changes to reflect updates. "One or more" servers, because for scalability and fault tolerance, big players such as Amazon and Google do not keep all data on one server.

- **It starts with a request:** the browser sends an HTTP request for a URL, often carrying query parameters or form data.
- **The server decides what's dynamic:** it hands the request to the web application rather than just returning a file. Not everything is dynamic: the page template stays the same, and only the parts that depend on the current state of the data are computed by the logic.
- **The application does the work:** querying a database if needed, then merging the results into an HTML template.
- **The response comes back:** as HTML, with a status code, **200** for success and **404** if nothing matched.
- **The browser then requests any CSS or JavaScript** the HTML references, usually served as **static files**.

Anatomy of a Request

Here the architecture is split across four separate actors, not one server: the **web server**, the **web application** (which is effectively the Controller and Model), and the **database**. The browser sends a GET request, the web server recognises that it is dynamic and forwards it to the application, and only the Model part of the application touches the database. A *read* operation fetches information (for example, which book); a *write* operation updates it (for example, a cart update). The data is merged into an HTML template, the finished page returns with its status code, and the browser parses it and asks for the linked CSS and JS.



Step	What happens	MVC link
1	Browser sends `GET /profile?id=42` (HTTP request)	User action reaches the system
2	Dynamic request is forwarded to the app	Hand-off to the Controller
3	Application queries the database for the needed records	Model accesses data
4	Records are returned	Model result
5	Data is merged into an HTML template	Controller and Model cooperate; View template is filled
6	Finished HTML returns (HTTP 200, or 404 if not found)	Response status chosen
7	HTML is delivered to the browser	View reaches the user's side
8	Browser parses the HTML and requests linked CSS/JS	Assets fetched

Static files (CSS, JS, images) are typically served directly by the web server, **without touching the app or the database**.

Practical Example: Tracing a Request

Say a student clicks **View Profile** on a course portal. Here is what happens in under a second:

- **Request:** the browser sends `GET /profile?id=42`.
- **Routing:** the web server sees that this is not a static file and hands it to the web application.
- **Data:** the application looks up student 42 in the database and merges the record into a template.
- **Response:** the finished HTML comes back with a **200 OK**, and the browser renders it.
- **Assets:** the page then requests `style.css` and `profile.js`, served directly by the web server.
- **If the student does not exist:** had student 42 not existed, the application would send back a **404 page** instead of a profile.

> **Before you code:** first think through the workflow, then segregate the concerns. Decide where the View, Model and Controller will reside. You may keep the Model and Controller on the server and only the View in the client, or keep some Controller logic in the client itself. This is a design decision to take *before* implementation, so take an example (for an assignment or project) and practise separating its concerns and modelling its workflow first.

Discussion: Alternatives, Trends and Limits

Other Alternatives

MVC is not the only option. Several architectures can work as alternatives or complements.

Alternative	Side	Idea	Where you see it
MVVM (Model–View–ViewModel)	Client	The Controller is replaced by a ViewModel; aims for a thin, "dumb" presentation layer that just presents what it is given	Vue, Angular
MVP (Model–View–Presenter)	Client	Similar to View plus Presenter; earlier Android apps used it	Desktop GUI frameworks
Component-based architecture	Client	The app is broken into disjoint components; each owns its own slice of Model, View and update logic (a mini-MVC inside each)	React
MVI (Model–View–Intent)	Client	The "Intent" models how the user interacts with UI elements and replaces the Controller's role	Android and reactive JS codebases
Microservices	Server	Many small services, each offering one specific function and interacting via APIs, often coordinated behind an **API gateway**	Large-scale back ends
Clean Architecture and **Hexagonal Architecture** (ports and adapters)	Server	Used within a single microservice to keep its core logic isolated	Newer service designs
CQRS (Command Query Responsibility Segregation)	Server	Separates commands (writes) from queries (reads)	Selected high-scale systems

Where Things Are Heading

React Server Components, **API-first** and **serverless** architectures, and **event-driven** architectures. This does **not** make MVC obsolete: it is still very much present and dominant in web applications. These are simply current trends in how the View and the Model/logic are segregated.

Limitations of MVC

- **The Controller tends to become a dumping ground.** All back-and-forth traffic between Model and View passes through it, which creates heavy overhead and pressure on it. This is one reason event-driven and serverless styles are gaining ground.
- **It adds real overhead for small applications**, where the separation is not earning its keep.
- **It does not map cleanly onto highly interactive, reactive UIs.** Routing every communication through the Controller adds latency, which hurts responsiveness in such setups.

> **Note for Review — Pro-Tip:** If asked "when is MVC a *bad* choice?", answer with the trade-off: separation costs overhead, so it is worth it as an application grows and is a poor fit for tiny apps and highly reactive UIs.

Recap

- MVC splits an application into a **Model (data + rules)**, a **View (display)**, and a **Controller (routes input to the other two)**.
- Keeping these three separate is **separation of concerns**: **high cohesion within each part, low coupling between them**.
- On the web, the frontend (HTML/CSS/JS in the browser) is mostly the **View**; the backend (server + database) usually holds the **Model** and much of the **Controller**. How you divide them depends on your application's scale and logic.
- A dynamic request is a conversation: browser to server, server to application, application to database, and back again as an HTML response.
- Most real frameworks such as Django, Rails and Express implement this pattern even when they never say "Model", "View" or "Controller" out loud, because it is deeply rooted in their architectural philosophy.

Practice Questions

1. **The greyed-out button.** An online store adds this requirement: *"An item with zero stock cannot be added to the cart, and its Add to Cart button must look greyed out."* Break the requirement into its parts and assign each part to Model, View or Controller. Then explain what would go wrong later if the stock check were written only inside the page's JavaScript, once the company also launches a mobile app on the same data.

2. **The 2,000-line Controller.** A teammate's controller file contains coupon calculations, hand-built HTML strings, and database queries. Diagnose which principles (cohesion, coupling, the roles of Model and View) are violated. Describe how you would refactor it, and name two concrete benefits the team would see on the day a designer redesigns the page *and* a database engineer changes the storage technology.

3. **Tracing the failure.** A browser requests `GET /profile?id=9999`, a student who does not exist. Walk through every component the request touches, say which one decides that the answer is a 404, and say whether the database is involved. Then, if the page's stylesheet link is broken, explain which component answers the CSS request and whether the application or database is involved. What would still work if the application crashed but the web server stayed alive?

4. **Live whiteboard.** You are building a collaborative whiteboard where every pen stroke must appear on all users' screens almost instantly. Argue whether strict Controller-mediated MVC (View → Controller → Model → Controller → View for each stroke) is a good fit. Identify which stated limitation matters most, then choose one alternative architecture from this chapter and justify what it gives you and what you would give up.

5. **Swapping the foundations.** A company moves from a relational database to a graph database and, at the same time, adds a VR front end alongside its web site. Which parts of the application should need *no* changes, if the concerns were separated well? What interface between the layers must stay stable for this to work? Give one realistic mistake (such as database logic leaking into a template) that would force changes on both sides, and explain why it breaks low coupling.